

Sardar Patel Institute of Technology
(Electronics & Telecommunication Department)



Mobile Wireless Communication

Experiment 3

Name: Rajas Mangirish Wagle

Class: TE(EXTC)

UID: 2023200129

Batch: B4

TITLE: Dynamic Channel Assignment Technique

AIM:

1. Assume Total Number of Channels(50-100) in a common pool.
2. Assume 10% of total Channels are reserved for Control channels. One Control Channel can be shared in two cells.
3. Remaining are Voice/Data channels.
4. Get input from users about cluster size and traffic channel demand per cell.
4. Distribute Control and Voice Channels in input size of Cluster as per request from user.
5. Display results of Control and Voice channels separately in matrix format.

THEORY:

Dynamic Channel Allocation:

In order to achieve optimum system capacity with limited frequency spectrum, many dynamic channel assignment schemes have been proposed to allocate the channels more efficiently. In a cellular system, a mobile subscriber moves from one cell to another, and continuation of communication link is ensured with a suitable hand-off mechanism. This demands for additional and flexible radio resources utilisation. One way to ascertain minimum blocking probability is to increase the number of channels per cell. Then every cell would expect to have a large number of channels. However, because a limited frequency band is allocated for cellular communication, there is an upper limit to the maximum number of channels, thereby restricting the number of available channels that can be assigned to each cell. Another way is nonuniform fixed channel assignment based on the amount of traffic expected to be served in different cells as per the statistical traffic data. Another alternative arrangement could be dynamic assignment of channels to different cells, as per the current demand. This may be done from a central pool of channels, or a combination of both fixed channel assignment and dynamic channel assignment. In dynamic channel assignment, the central common pool maintains all the available channels. Channels are assigned dynamically as new requests for radio resources (for a fresh originating call or hand-off of existing call) arrive in the system. This also implies that when the use of the assigned channel is completed, the channel currently in use is returned to the central pool.

CODE:

```
import random

def dynamic_channel_allocation():
    # taking input for total number of channels, show error if index is
    out of bounds (!(50-100))
    total_channels = int(input("Enter total number of channels (50-100):
    "))
    if total_channels < 50 or total_channels > 100:
        print("Number of channels must be between 50 and 100.")
        return

    # 10% of total channels is allocated for control channels
    control_channels = total_channels // 10
    voice_channels = total_channels - control_channels
    # display number of channels left
    print(f"\nTotal Channels: {total_channels}")
    print(f"Control Channels Reserved: {control_channels}")
    print(f"Voice/Data Channels Available: {voice_channels}")

    # taking input for cluster size, keep asking until it equals 7
    while True:
        cluster_size = int(input("\nEnter cluster size: "))
        if cluster_size == 7:
            break
        else:
            print("Error: Cluster size must be valid.")

    # creating list to store demands demanded by user per cluster.
    demand_list = []
    print(f"\nEnter channel demand for {cluster_size} cells:")
    for i in range(cluster_size):
        demand = int(input(f" Enter demand for Cell {i+1}: "))
        demand_list.append(demand)
```

```

# creating copy of demand list for further calculation
original_demand_list = demand_list[:]

# summing all demands to keep a tab of total demands for percentage
difference
total_demand_requested = sum(original_demand_list)

# Create individual channel pool
channel_pool = list(range(1, voice_channels + 1))

# Assign random priority channel wise
channel_priorities = {}
for ch in channel_pool:
    p = random.randint(1, 5)
    if p in [1, 2]:
        channel_priorities[ch] = ("L", 1)
    elif p == 3:
        channel_priorities[ch] = ("M", 2)
    else:
        channel_priorities[ch] = ("H", 3)

total_demand = sum(demand_list)
allocations = [[] for _ in range(cluster_size)]

if total_demand > voice_channels:
    print(f"\nTotal demand {total_demand} exceeds available
{voice_channels}. Adjusting...")

shares = [0] * cluster_size
remaining_channels = voice_channels

for i in range(cluster_size):
    if demand_list[i] < 10:
        shares[i] = demand_list[i]
        remaining_channels -= demand_list[i]

```

```

        large_demand_total = sum(d for d in demand_list if d >= 10)
        for i in range(cluster_size):
            if demand_list[i] >= 10:
                share = int((demand_list[i] / large_demand_total) *
remaining_channels)
                shares[i] = share

        leftover = remaining_channels - sum(shares[i] for i in
range(cluster_size) if demand_list[i] >= 10)
        i = 0
        while leftover > 0:
            if demand_list[i] >= 10:
                shares[i] += 1
                leftover -= 1
            i = (i + 1) % cluster_size

        # Final allocation
        for i in range(cluster_size):
            req = shares[i]
            allocations[i] = channel_pool[:req]
            channel_pool = channel_pool[req:]

    else:
        for i in range(cluster_size):
            req = demand_list[i]
            allocations[i] = channel_pool[:req]
            channel_pool = channel_pool[req:]

    allocated_total = sum(len(a) for a in allocations)
    remaining = voice_channels - allocated_total

    print("\nFinal Voice/Data Channel allocation matrix:")
    for i, channels in enumerate(allocations, start=1):
        labeled = [(ch, channel_priorities[ch][0]) for ch in channels]
        print(f"    Cell {i}: {labeled}")

    print(f"\nTotal allocated: {allocated_total} (of {voice_channels})")

```

```

print(f"Remaining unallocated channels: {remaining}")

# list for high medium low channels
high_channels = []
medium_channels = []
low_channels = []

for cell in allocations:
    high_channels.extend([ch for ch in cell if
channel_priorities[ch][0] == "H"])
    medium_channels.extend([ch for ch in cell if
channel_priorities[ch][0] == "M"])
    low_channels.extend([ch for ch in cell if
channel_priorities[ch][0] == "L"])

print("\nHigh Priority Channels:")
print(high_channels)

print("\nMedium Priority Channels:")
print(medium_channels)

print("\nLow Priority Channels:")
print(low_channels)

# Blocking Probability
print("\nBlocking Probability per Cell:")
for i in range(cluster_size):
    demand = original_demand_list[i]
    allocated = len(allocations[i])
    if demand > 0:
        blocking_prob = ((demand - allocated) / demand) * 100
        print(f"  Cell {i+1}: Demand={demand}, Allocated={allocated},
Blocking Probability={blocking_prob:.6f}%")
    else:
        print(f"  Cell {i+1}: Demand=0, Allocated={allocated},
Blocking Probability=N/A")

```

```

# Control Channel Allocation (each shared by 2 cells)
print("\nControl Channel Allocation:")
control_pool = list(range(total_channels - control_channels + 1,
total_channels + 1)) # e.g., 91-100
control_allocations = [[] for _ in range(cluster_size)]

i = 0
while control_pool:
    ch = control_pool.pop(0)
    control_allocations[i % cluster_size].append(ch)
    control_allocations[(i + 1) % cluster_size].append(ch)
    i += 2

for i, chans in enumerate(control_allocations, start=1):
    print(f"  Cell {i}: {chans}")

dynamic_channel_allocation()

```

OUTPUT(S):

1. For Cluster size != 7, the program throws an input error.

```

Enter total number of channels (50-100): 100

Total Channels: 100
Control Channels Reserved: 10
Voice/Data Channels Available: 90

Enter cluster size: 9
Error: Cluster size must be valid.

Enter cluster size: 7

Enter channel demand for 7 cells:

```

2. Now, for a valid input for cluster size of 7, 100 total channels, demand > total available :

```
Enter total number of channels (50-100): 100
Total Channels: 100
Control Channels Reserved: 10
Voice/Data Channels Available: 90

Enter cluster size: 9
Error: Cluster size must be valid.

Enter cluster size: 7

Enter channel demand for 7 cells:
Enter demand for Cell 1: 61
Enter demand for Cell 2: 7
Enter demand for Cell 3: 2
Enter demand for Cell 4: 34
Enter demand for Cell 5: 25
Enter demand for Cell 6: 31
Enter demand for Cell 7: 10

Total demand 170 exceeds available 90. Adjusting...

Final Voice/Data Channel allocation matrix:
Cell 1: [(1, 'H'), (2, 'H'), (3, 'L'), (4, 'H'), (5, 'L'), (6, 'H'), (7, 'L'), (8, 'L'), (9, 'L'), (10, 'L'), (11, 'H'), (12, 'L'), (13, 'H'), (14, 'H'), (15, 'L'), (16, 'H'), (17, 'L'), (18, 'H'), (19, 'L'), (20, 'L'), (21, 'L'), (22, 'L'), (23, 'L'), (24, 'L'), (25, 'L'), (26, 'L'), (27, 'L'), (28, 'L'), (29, 'L'), (30, 'L'), (31, 'L'), (32, 'L'), (33, 'L'), (34, 'M'), (35, 'M'), (36, 'L'), (37, 'H'), (38, 'H')]
Cell 2: [(39, 'L'), (40, 'L')]
Cell 3: [(41, 'H'), (42, 'H'), (43, 'H'), (44, 'H'), (45, 'H'), (46, 'L'), (47, 'L'), (48, 'H'), (49, 'L'), (50, 'L'), (51, 'H'), (52, 'H'), (53, 'H'), (54, 'H'), (55, 'H'), (56, 'H'), (57, 'L'), (58, 'H')]
Cell 4: [(59, 'H'), (60, 'H'), (61, 'M'), (62, 'L'), (63, 'L'), (64, 'M'), (65, 'L'), (66, 'L'), (67, 'M'), (68, 'L'), (69, 'H'), (70, 'L')]
Cell 5: [(71, 'H'), (72, 'H'), (73, 'L'), (74, 'H'), (75, 'H'), (76, 'L'), (77, 'H'), (78, 'H'), (79, 'L'), (80, 'H'), (81, 'L'), (82, 'L'), (83, 'L'), (84, 'H'), (85, 'H')]
Cell 6: [(86, 'H'), (87, 'L'), (88, 'L'), (89, 'H'), (90, 'H')]

Total allocated: 90 (of 90)
Remaining unallocated channels: 0

High Priority Channels:
[1, 2, 4, 13, 14, 18, 29, 30, 37, 38, 43, 44, 45, 51, 53, 54, 55, 56, 59, 69, 71, 72, 74, 75, 77, 80, 90]

Medium Priority Channels:
[6, 11, 16, 28, 34, 35, 41, 42, 48, 52, 58, 60, 61, 64, 67, 78, 84, 85, 86, 89]

Low Priority Channels:
[3, 5, 7, 8, 9, 10, 12, 15, 17, 19, 20, 21, 22, 23, 24, 25, 26, 27, 31, 32, 33, 36, 39, 40, 46, 47, 49, 50, 57, 62, 63, 65, 66, 68, 70, 73, 76, 79, 81, 82, 83, 87, 88]

Blocking Probability per Cell:
Cell 1: Demand=61, Allocated=31, Blocking Probability=49.180328%
Cell 2: Demand=7, Allocated=7, Blocking Probability=0.000000%
Cell 3: Demand=2, Allocated=2, Blocking Probability=0.000000%
Cell 4: Demand=34, Allocated=18, Blocking Probability=47.058824%
Cell 5: Demand=25, Allocated=12, Blocking Probability=52.000000%
Cell 6: Demand=31, Allocated=15, Blocking Probability=51.612903%
Cell 7: Demand=10, Allocated=5, Blocking Probability=50.000000%

Control Channel Allocation:
Cell 1: [91, 94, 98]
Cell 2: [91, 95, 98]
Cell 3: [92, 95, 99]
Cell 4: [92, 96, 99]
Cell 5: [93, 96, 100]
Cell 6: [93, 97, 100]
Cell 7: [94, 97]
```

By observation, the program assigns priorities to channels, between 1-5 wherein,
1-2 :: LOW
3 :: Medium
4-5 :: High.

We calculate the blocking probability as well for each individual cell, based on cell demand inputted by the user.

If demand of cell (N) <10, all of the channels demanded are allotted randomly (ex: if user enters 8 as demand for cell 3, cell 3 will get 8/8 channels 100%, no reduction, hence blocking probability = 0 for such cases, as shown above.

3. Now, for a valid input for cluster size of 7, 100 total channels, demand $\leq$ total available :

```
Enter total number of channels (50-100): 100
Total Channels: 100
Control Channels Reserved: 10
Voice/Data Channels Available: 90

Enter cluster size: 8
Error: Cluster size must be valid.

Enter cluster size: 7

Enter channel demand for 7 cells:
Enter demand for Cell 1: 10
Enter demand for cell 2: 10
Enter demand for cell 3: 10
Enter demand for Cell 4: 23
Enter demand for Cell 5: 7
Enter demand for cell 6: 6
Enter demand for Cell 7: 5

Final Voice/Data Channel allocation matrix:
Cell 1: [(1, 'H'), (2, 'M'), (3, 'L'), (4, 'H'), (5, 'H'), (6, 'H'), (7, 'H'), (8, 'H'), (9, 'H'), (10, 'L')]
Cell 2: [(11, 'H'), (12, 'L'), (13, 'H'), (14, 'L'), (15, 'H'), (16, 'H'), (17, 'L'), (18, 'H'), (19, 'H'), (20, 'L')]
Cell 3: [(21, 'L'), (22, 'M'), (23, 'M'), (24, 'L'), (25, 'H'), (26, 'H'), (27, 'L'), (28, 'H'), (29, 'M'), (30, 'M')]
Cell 4: [(31, 'H'), (32, 'L'), (33, 'H'), (34, 'L'), (35, 'M'), (36, 'M'), (37, 'H'), (38, 'H'), (39, 'H'), (40, 'L'), (41, 'L'), (42, 'H'), (43, 'M'), (44, 'M'), (45, 'L'), (46, 'M'), (47, 'L'), (48, 'H'), (49, 'H'), (50, 'L'), (51, 'L')]
Cell 5: [(54, 'M'), (55, 'H'), (56, 'M'), (57, 'H'), (58, 'M'), (59, 'H'), (60, 'L')]
Cell 6: [(61, 'H'), (62, 'L'), (63, 'M'), (64, 'M'), (65, 'L'), (66, 'H')]
Cell 7: [(67, 'M'), (68, 'L'), (69, 'H'), (70, 'L'), (71, 'L')]

Total allocated: 71 (of 90)
Remaining unallocated channels: 19
```

```
High Priority Channels:
[1, 5, 6, 7, 8, 11, 13, 16, 25, 26, 28, 31, 33, 37, 38, 39, 42, 40, 55, 57, 58, 59, 61, 66, 69]

Medium Priority Channels:
[2, 4, 9, 15, 18, 19, 22, 23, 29, 30, 35, 36, 43, 44, 46, 48, 54, 56, 63, 64, 67]

Low Priority Channels:
[3, 10, 12, 14, 17, 20, 21, 24, 27, 32, 34, 40, 41, 45, 47, 50, 51, 52, 53, 60, 62, 65, 68, 70, 71]

Blocking Probability per Cell:
Cell 1: Demand=10, Allocated=10, Blocking Probability=0.000000%
Cell 2: Demand=10, Allocated=10, Blocking Probability=0.000000%
Cell 3: Demand=10, Allocated=10, Blocking Probability=0.000000%
Cell 4: Demand=23, Allocated=23, Blocking Probability=0.000000%
Cell 5: Demand=7, Allocated=7, Blocking Probability=0.000000%
Cell 6: Demand=6, Allocated=6, Blocking Probability=0.000000%
Cell 7: Demand=5, Allocated=5, Blocking Probability=0.000000%

Control Channel Allocation:
Cell 1: [91, 94, 98]
Cell 2: [91, 95, 98]
Cell 3: [92, 95, 99]
Cell 4: [92, 96, 99]
Cell 5: [93, 96, 100]
Cell 6: [93, 97, 100]
Cell 7: [94, 97]
```

Here since demand(net)<Total available channels, 71<90,
All channels are allocated as per demand, cell wise, yet randomly.

REFERENCE(s): Wireless Communication - Tata McGraw Hill(3rd Edition)

CONCLUSION:

I have successfully implemented a simulation for a Dynamic Channel Allocation(DCA) setup in Python. I have read through the literature for Dynamic Channel Allocation techniques, and hence have come to a conclusion that DCA is better than FCA for a network for a large amount of users like 4G/5G cellular subscribers for a metropolitan area like Mumbai.

It also has its respective disadvantages, like high computational complexity, it requires real-time monitoring of traffic patterns, interference levels, and channel availability. This increases the processing load compared to fixed channel allocation (FCA).